

CENTRO UNIVERSITÁRIO - CATÓLICA DE SANTA CATARINA
ENGENHARIA DE SOFTWARE
PROJETO DE APRENDIZAGEM COLABORATIVA

GUILHERME RICARDO MACHADO

PLANIT GO

JOINVILLE
2025

RESUMO

Este documento apresenta o desenvolvimento de um Web App de Planejamento de Viagens, focado no controle de orçamento e organização de itinerário para viajantes. O objetivo principal do projeto é criar uma ferramenta acessível e prática para que os usuários possam planejar suas viagens de forma eficiente, controlando seus gastos e ajustando o planejamento conforme necessário. Através de uma interface simples, a solução permite o gerenciamento do orçamento em categorias como transporte, alimentação, hospedagem e atividades, além de fornecer relatórios ao final da viagem para análise do planejamento versus os gastos reais. Este projeto busca resolver a dificuldade de muitos viajantes em organizar suas despesas e planejar suas atividades sem depender de ferramentas excessivamente complexas.

Palavras-chave: Planejamento de Viagens. Controle de Orçamento. Organização de Itinerário. Relatórios de Viagem. Usuários.

SUMÁRIO

1	INTRODUÇÃO	4
1.1	CONTEXTO	4
1.2	JUSTIFICATIVA	4
1.3	OBJETIVOS	5
2	DESCRIÇÃO DO PROJETO	6
2.1	LINHA DE PROJETO	6
2.2	TEMA DO PROJETO	6
2.3	PROPÓSITO E USO PRÁTICO	6
2.4	PÚBLICO-ALVO	6
2.5	PROBLEMAS A RESOLVER	6
2.6	DIFERENCIAÇÃO/INEDITISMO	7
2.7	LIMITAÇÕES	7
2.8	NORMAS E LEGISLAÇÕES APLICÁVEIS	7
2.9	MÉTRICAS DE SUCESSO	7
3	ESPECIFICAÇÕES TÉCNICAS	9
3.1	REQUISITOS DE SOFTWARE	9
3.1.1	Requisitos Funcionais (RF)	9
3.1.2	Requisitos Não-Funcionais (RNF)	9
3.1.3	Representação dos Requisitos	10
3.1.4	Aderência aos Requisitos da Linha de Projeto	10
3.2	CONSIDERAÇÕES DE DESIGN	11
3.2.1	Visão Inicial da Arquitetura	11
3.2.2	Padrões de Arquitetura	11
3.2.3	Modelo Entidade Relacionamento	12
3.2.4	Diagramas	12
3.2.5	Mockups das Telas Principais	13
3.2.6	Decisões e Alternativas Consideradas	14
3.2.7	Critérios de Escalabilidade, Resiliência e Segurança	14
3.3	FERRAMENTAS TECNOLÓGICAS	15
3.3.1	Linguagens de Programação	15
3.3.2	Frameworks e Bibliotecas	15
3.3.3	Ferramentas de Desenvolvimento e Gestão	16
3.3.4	Licenciamento	17
3.4	CONSIDERAÇÕES DE SEGURANÇA	17
3.4.1	Riscos Identificados	17

3.4.2	Medidas de Mitigação	18
3.4.3	Normas e Boas Práticas Seguidas	18
3.4.4	Responsabilidade Ética	18
3.5	CONFORMIDADE E NORMAS APLICÁVEIS	18
4	PRÓXIMOS PASSOS	20
4.0.1	Marcos de Entrega (Portfólio I)	20
4.0.2	Próximos Passos e Visão Geral do Portfólio II	20
	REFERÊNCIAS	21

1 INTRODUÇÃO

O planejamento de viagens é uma atividade comum, mas frequentemente desafiadora, especialmente quando envolve o controle financeiro. Muitas pessoas enfrentam dificuldades ao tentar equilibrar o orçamento, gerenciar os gastos durante a viagem e criar itinerários eficientes que atendam às suas necessidades. A gestão de despesas durante viagens é um problema bem documentado, com estudos indicando que muitas pessoas extrapolam seus orçamentos ou têm dificuldades em manter o controle durante a viagem (Experian, 2025). Embora existam diversas ferramentas no mercado, muitas oferecem conjuntos amplos de funcionalidades, o que pode afastar quem busca uma solução prática e direta. Exemplos: TripIt, focado em consolidar e organizar reservas de múltiplas fontes, e Wanderlog, que adiciona recursos de planejamento colaborativo e orçamento.

1.1 CONTEXTO

O projeto está inserido em um cenário de crescimento do turismo nacional e de forte presença de soluções digitais no planejamento de viagens. Em 2024, os gastos com viagens nacionais com pernoite atingiram R\$ 22,8 bilhões, um aumento de 11,7% em relação a 2023, o que reforça a relevância econômica das viagens para os consumidores brasileiros (Estatística, 2025).

Paralelamente, conteúdos de educação financeira voltados ao público que deseja viajar destacam que transformar o desejo de viagem em realidade exige estimar e organizar todos os custos envolvidos — transporte, hospedagem, alimentação, passeios e imprevistos —, geralmente com apoio de planilhas e ferramentas específicas para controle de orçamento (Sene, 2025). Esse contexto, marcado pelo aumento do turismo e pela digitalização do planejamento, evidencia a importância de soluções web que apoiem o viajante na organização de informações e no acompanhamento do orçamento ao longo da viagem.

1.2 JUSTIFICATIVA

Diante de todo o cenário, este projeto se justifica pela necessidade de reduzir a fragmentação do planejamento de viagens para usuários que já utilizam recursos digitais, mas ainda precisam alternar entre diversos sites e planilhas para organizar gastos e atividades. Ao propor um web app que centraliza o planejamento do itinerário e o controle financeiro em uma única ferramenta simples e acessível, o projeto busca facilitar o acompanhamento do orçamento ao longo da viagem e apoiar decisões mais conscientes de uso do dinheiro. Além disso, representa uma oportunidade de aplicação prática dos conceitos de engenharia de software estudados no curso, conectando um problema real do cotidiano a uma solução tecnológica concreta.

1.3 OBJETIVOS

Os objetivos definem o que o projeto pretende alcançar. Podem ser divididos em:

- **Objetivo Principal:** Desenvolver uma ferramenta para o planejamento de viagens, com foco em controle de orçamento e organização de itinerário.
- **Objetivos Secundários:**
 - Configurar o ambiente de desenvolvimento, repositório de código e pipeline de integração contínua do projeto.
 - Implementar o módulo de cadastro de usuários e autenticação para acesso ao sistema.
 - Desenvolver o módulo de planejamento de viagens, permitindo o registro de destino, datas e orçamento.
 - Implementar o módulo de controle orçamentário, com distribuição de gastos entre categorias como transporte, alimentação, hospedagem e atividades.
 - Desenvolver a funcionalidade de acompanhamento de gastos e visualização do itinerário durante a viagem.
 - Implementar a geração de relatórios para comparação entre orçamento planejado e gastos realizados.
 - Disponibilizar a aplicação como Progressive Web App (PWA) em ambiente de nuvem pública.

2 DESCRIÇÃO DO PROJETO

2.1 LINHA DE PROJETO

Web App com PWA.

Uma Progressive Web App (PWA, "Aplicação Web Progressiva") é uma aplicação criada com tecnologias web, mas que oferece uma experiência de usuário como a de uma específica de plataforma (Docs, 2025).

2.2 TEMA DO PROJETO

O projeto consiste no desenvolvimento de um web app chamado *Planit Go*, voltado ao planejamento de viagens personalizadas para viajantes que organizam suas viagens de forma independente, com foco no controle orçamentário e no acompanhamento dos gastos.

2.3 PROPÓSITO E USO PRÁTICO

O propósito do projeto é resolver a dificuldade que muitos viajantes enfrentam ao tentar planejar viagens com orçamento limitado. A solução permitirá que o usuário distribua seu orçamento entre categorias essenciais e acompanhe os gastos em tempo real, garantindo que o planejamento financeiro seja respeitado. A plataforma será utilizada de forma intuitiva, sem complexidade, permitindo que o viajante controle sua viagem de maneira eficaz, mesmo sem experiência prévia no gerenciamento de orçamento.

2.4 PÚBLICO-ALVO

- Viajantes independentes, como jovens adultos, famílias pequenas e profissionais que buscam planejar suas viagens sem dependência de agências.
- Pessoas com orçamento limitado, que necessitam de uma ferramenta simples para garantir que não ultrapassem o valor planejado durante a viagem.
- Turistas ocasionais e viagens curtas (fins de semana ou feriados), que buscam uma solução prática e eficaz para o controle financeiro de sua viagem.

2.5 PROBLEMAS A RESOLVER

- Dificuldade de distribuir o orçamento de maneira equilibrada entre as categorias essenciais.
- Dificuldade em manter o controle financeiro sem precisar de plataformas complexas
- Falta de visibilidade sobre como os gastos reais se comparam com o orçamento planejado.

2.6 DIFERENCIAÇÃO/INEDITISMO

O projeto se diferencia por oferecer uma solução prática e autossuficiente, com foco no controle financeiro e organização simples de itinerários. Ele oferece distribuição automática do orçamento nas principais categorias de viagem, um acompanhamento de gastos em tempo real, e relatórios de comparação entre o planejado e o real, tornando-o único em relação às soluções existentes, que muitas vezes são excessivamente complexas ou focam apenas na organização de itinerários, sem dar atenção ao controle financeiro.

2.7 LIMITAÇÕES

O projeto não incluirá integrações com sistemas externos, como APIs de voos e hotéis, por exemplo. A proposta é criar uma ferramenta autossuficiente, sem depender de dados externos, garantindo uma experiência mais simples e focada no planejamento financeiro.

2.8 NORMAS E LEGISLAÇÕES APLICÁVEIS

LGPD (Lei Geral de Proteção de Dados – Lei nº 13.709/2018): O sistema coletará apenas os dados pessoais necessários para o funcionamento da aplicação (como nome, e-mail e credenciais de acesso). No momento do cadastro, o usuário deverá aceitar um termo de consentimento para o tratamento desses dados, com acesso a uma página de Política de Privacidade. Será disponibilizada uma área de perfil para visualização e atualização dos dados cadastrados, bem como a opção de exclusão da conta, removendo os dados pessoais do usuário do banco de dados.

Marco Civil da Internet (Lei nº 12.965/2014): Serão observados princípios de proteção à privacidade e à segurança dos dados de acesso ao sistema, utilizando comunicação via HTTPS, armazenamento de senhas em formato criptografado (hash) e guarda responsável de registros mínimos de acesso, apenas pelo tempo necessário ao funcionamento e suporte da aplicação.

2.9 MÉTRICAS DE SUCESSO

As métricas abaixo serão utilizadas como critérios iniciais para avaliar o sucesso do Planit Go, alinhadas a recomendações de qualidade de software em termos de funcionalidade, desempenho e teste (Pressman; Maxim, 2016; BrowserStack, 2025):

- Cobertura de requisitos implementados: percentual de requisitos funcionais definidos na RFC que foram efetivamente implementados e validados por meio de casos de teste.
- Corretude do controle de orçamento: quantidade de casos de teste bem-sucedidos envolvendo criação de orçamento, registro de despesas e cálculo de saldo restante. Serão definidos cenários de teste com valores de entrada e resultados esperados, verificando se o sistema apresenta corretamente totais e diferenças entre orçamento planejado e gastos registrados.

- Desempenho da aplicação: tempo de carregamento das principais telas (login, tela inicial, criação de viagem e visão de orçamento) e tempo de resposta para operações críticas, medidos com as ferramentas de desenvolvimento do navegador em diferentes execuções.
- Estabilidade durante o uso: número de erros ou falhas (bugs) observados durante a execução dos testes funcionais e exploratórios do sistema, considerando especialmente operações de cadastro de viagem, registro de gastos e visualização do itinerário.

3 ESPECIFICAÇÕES TÉCNICAS

Descrição detalhada da proposta, contemplando requisitos, arquitetura, tecnologias, segurança e aderência aos critérios obrigatórios da linha de projeto escolhida.

3.1 REQUISITOS DE SOFTWARE

3.1.1 Requisitos Funcionais (RF)

- **RF01: Cadastro de Viagens:** O sistema deve permitir o cadastro de uma nova viagem, incluindo **destino, datas e orçamento**.
- **RF02: Distribuição do Orçamento:** O sistema deve sugerir uma **distribuição automática do orçamento** entre categorias (transporte, alimentação, hospedagem, atividades). 
- **RF03: Controle de Gastos:** O sistema deve permitir o registro de **gastos reais** e a comparação com o **orçamento planejado**. 
- **RF04: Criação de Itinerário:** O usuário deve poder criar um **itinerário diário** com atividades e eventos relacionados à viagem.
- **RF05: Relatórios de Planejamento:** O sistema deve gerar um **relatório** final com a comparação entre o **orçamento** e o **gasto real**.
- **RF06: Alertas de Excesso de Gasto:** O sistema deve notificar o usuário quando os gastos de uma categoria estiverem **se aproximando do limite**.
- **RF07: Armazenamento de Viagens:** O usuário deve poder **salvar e consultar viagens anteriores**, incluindo orçamentos e itinerários.
- **RF08: Disponibilidade Mobile:** A aplicação deve ser instalável como um **PWA**, permitindo ao usuário adicionar à tela inicial e utilizar offline.

3.1.2 Requisitos Não-Funcionais (RNF)

- **RNF01 - Desempenho:** As principais telas do sistema (login, página inicial, criação de viagem e visão de orçamento) devem responder de forma fluida, sem travamentos ou esperas longas que atrapalhem o uso. Durante os testes, os tempos de carregamento e resposta serão observados pelas ferramentas de desenvolvimento do navegador. 
- **RNF02 - Segurança:** As senhas dos usuários devem ser armazenadas de forma segura (hash) e o acesso ao sistema será protegido por um mecanismo de autenticação com tokens (por exemplo, JWT). No ambiente de produção, a aplicação deve ser acessível preferencialmente via HTTPS, conforme os recursos oferecidos pelo provedor de hospedagem. 

- **RNF03 - Usabilidade:** A interface deve ser simples e intuitiva, com telas organizadas, textos claros e feedback visual para as ações importantes do usuário, como salvar dados, validar campos obrigatórios e exibir mensagens de erro.
- **RNF04 - Disponibilidade e implantação:** O sistema deve ser publicado em um serviço de hospedagem em nuvem, permitindo que o Planit Go seja acessado pelo navegador, sem necessidade de instalação local.
- **RNF05 - Compatibilidade:** O sistema deve funcionar nos principais navegadores modernos, incluindo Google Chrome, Mozilla Firefox e Microsoft Edge em suas versões estáveis, com visualização adequada em telas de desktop e dispositivos móveis.
- **RNF06 - Acessibilidade:** A interface deve ser clara e legível, usando textos compreensíveis, boa organização visual das informações e rótulos objetivos em botões e campos, de modo a facilitar o uso por diferentes tipos de usuários.

3.1.3 Representação dos Requisitos

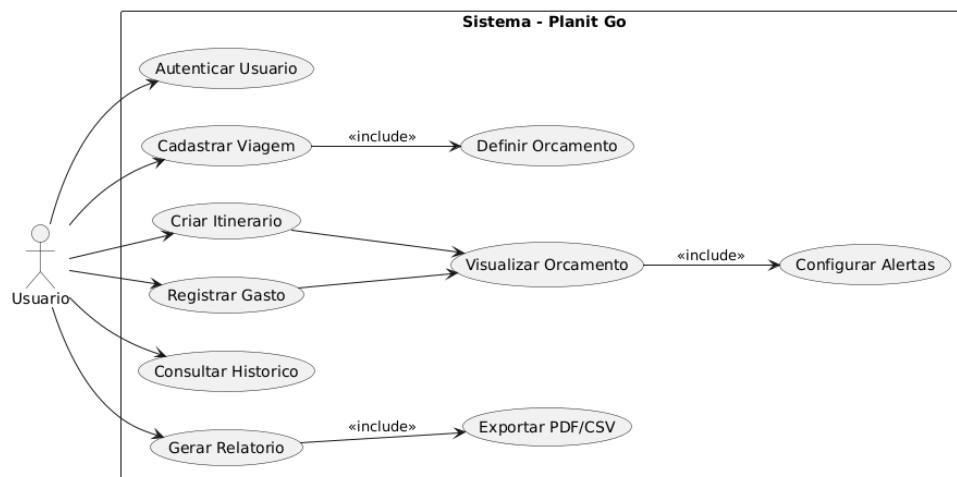


Figura 1 – Diagrama de casos de uso do sistema Planit Go

3.1.4 Aderência aos Requisitos da Linha de Projeto

- **Hospedagem pública:** O projeto será hospedado em plataforma pública ainda não definida, garantindo alta disponibilidade e deploy contínuo a partir do repositório de código.
- **Funcionalidades completas conforme escopo:** O sistema implementa as funcionalidades essenciais de controle de orçamento, criação de itinerários, registro de gastos, e geração de relatórios conforme o escopo definido no RFC.

- **Interface funcional e navegável:** A interface foi projetada para ser intuitiva e de fácil navegação, com usabilidade básica, permitindo ao usuário realizar todas as operações necessárias de forma simples e eficiente.
- **Arquitetura de software definida:** A arquitetura do sistema segue o padrão MVC (Model-View-Controller), garantindo uma separação clara entre a lógica de negócios, interação com o banco de dados e interface do usuário, o que facilita a manutenção e escalabilidade.

3.2 CONSIDERAÇÕES DE DESIGN

3.2.1 Visão Inicial da Arquitetura

A arquitetura inicial do Planit Go seguirá o modelo cliente-servidor, organizada em camadas. De forma geral, o sistema será composto por três blocos principais:

- **Camada de apresentação (cliente web/PWA):** interface acessada pelo navegador, responsável pela interação com o usuário. Nessa camada o usuário poderá cadastrar viagens, registrar gastos, visualizar o orçamento e consultar relatórios, utilizando uma aplicação web responsiva que poderá ser instalada como PWA.
- **Camada de aplicação (API do sistema):** serviço web responsável pelas regras de negócio do Planit Go. Essa camada receberá as requisições da interface (por exemplo, criar viagem, registrar gasto, consultar histórico), fará as validações necessárias, aplicará as regras de cálculo de orçamento e organizará os dados a serem enviados de volta ao cliente.
- **Camada de dados (banco de dados):** responsável pelo armazenamento persistente das informações de usuários, viagens, itinerários, categorias de gasto e registros de despesas. A camada de aplicação será a única a acessar diretamente o banco de dados, mantendo o cliente desacoplado dos detalhes de persistência.

A interação entre os componentes ocorrerá por meio de requisições HTTP, em que a camada de apresentação enviará e receberá dados em formato estruturado (por exemplo, JSON) da camada de aplicação.

3.2.2 Padrões de Arquitetura

O projeto adota o padrão Model-View-Controller (MVC), que separa claramente as responsabilidades entre lógica de negócios (Model), interface com o usuário (View) e controle das interações (Controller). Esse padrão facilita a manutenção e escalabilidade do sistema, permitindo a modificação independente de cada componente.

3.2.3 Modelo Entidade Relacionamento

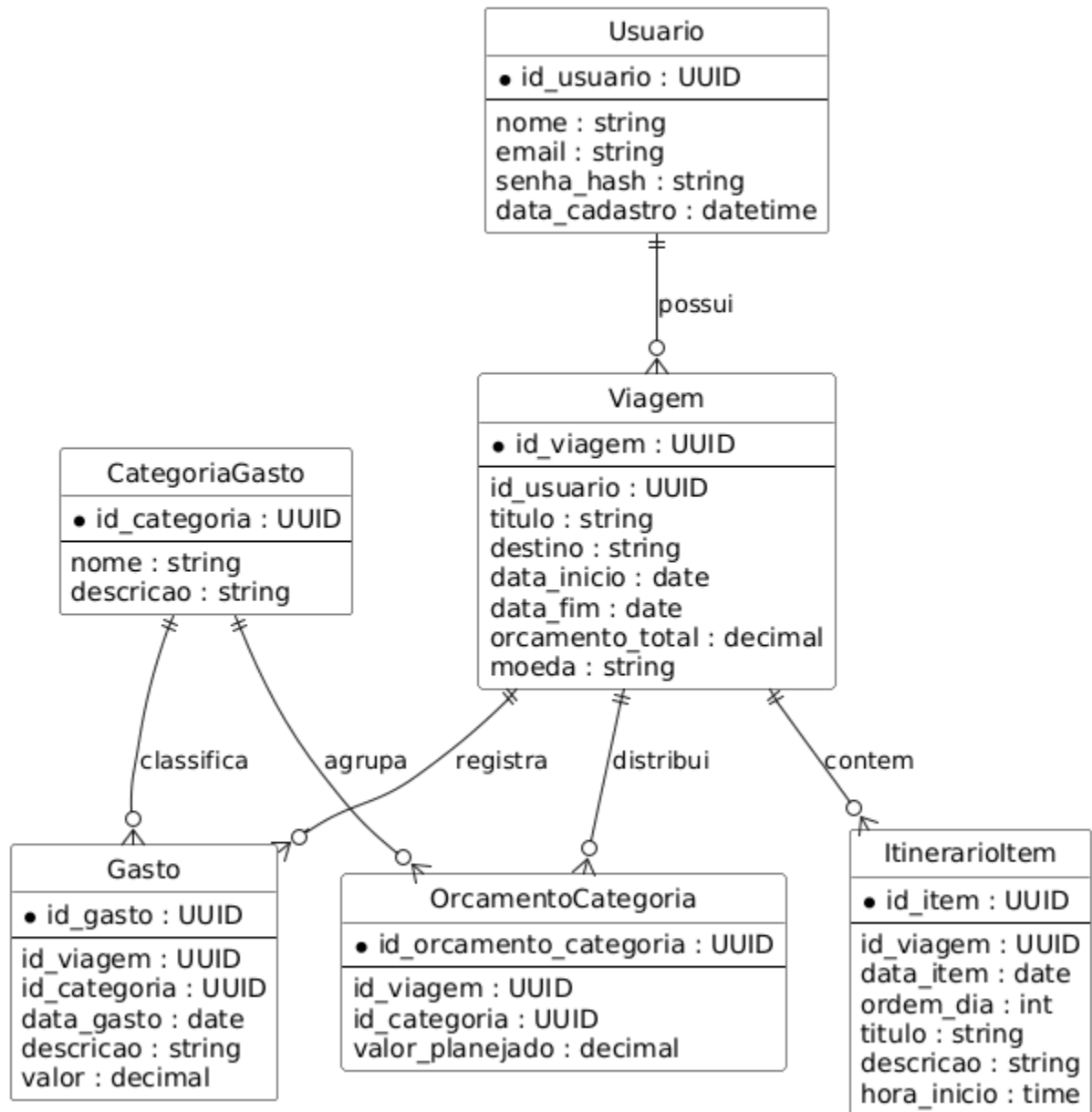


Figura 2 – Modelo entidade-relacionamento do sistema Planit Go

3.2.4 Diagramas

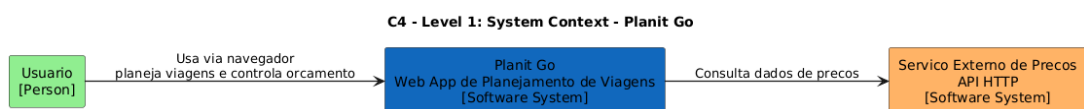


Figura 3 – C4 Level 1 – Diagrama de contexto do sistema Planit Go

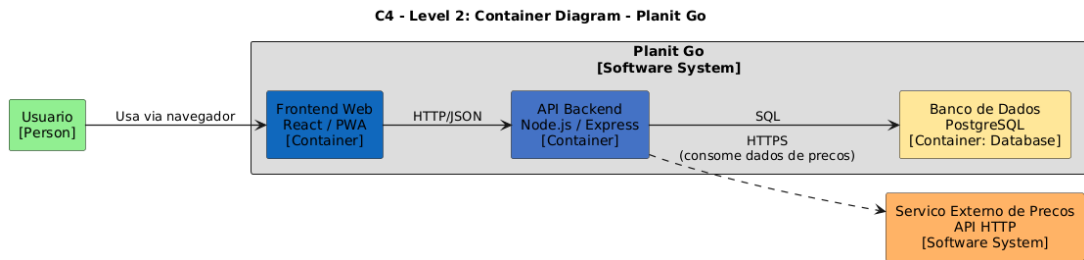


Figura 4 – C4 Level 2 – Diagrama de containers do sistema Planit Go

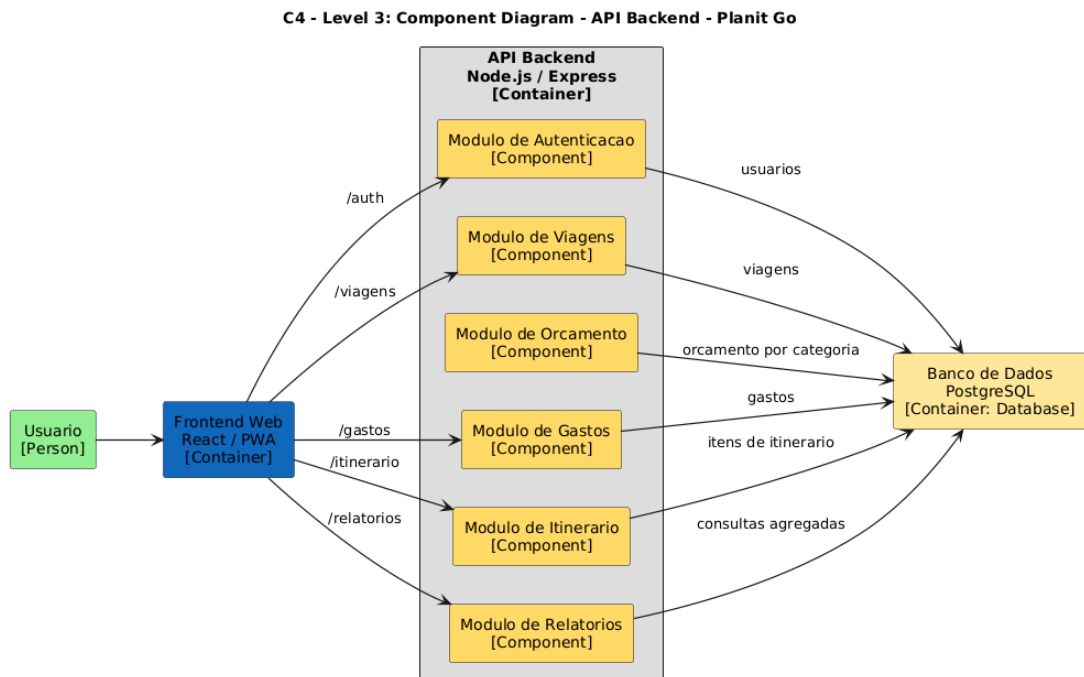


Figura 5 – C4 Level 3 – Diagrama de componentes da API backend do sistema Planit Go



3.2.5 Mockups das Telas Principais

Planit Go
Organize sua viagem de acordo com seu orçamento.

Nova Viagem

Destino: Ex: Paris País: Ex: França

Data de Início: dd/mm/aaaa Data de Término: dd/mm/aaaa

Orçamento (R\$): Ex: 5000

URL da Imagem (opcional): https://exemplo.com/imagem.jpg

Descrição: Descreva sua viagem...

Cancelar Criar Viagem

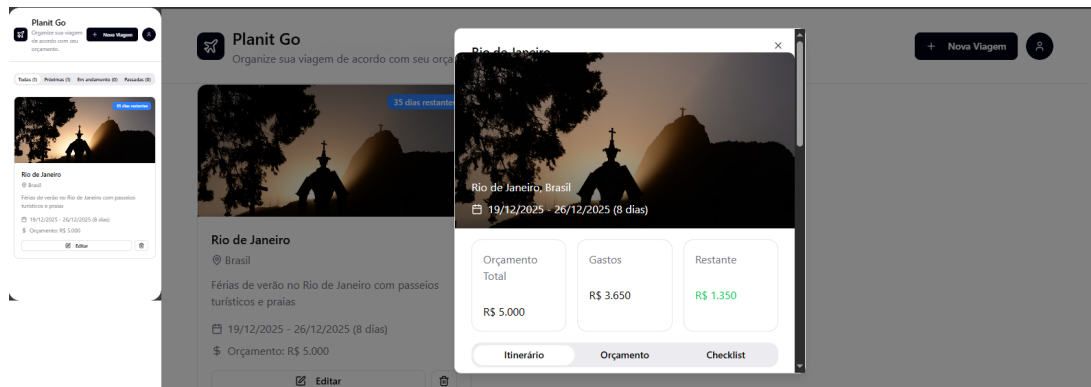


Figura 6 – Mockups das telas principais do sistema Planit Go

3.2.6 Decisões e Alternativas Consideradas

Algumas decisões de design de interface e experiência do usuário foram tomadas para manter o Planit Go simples e objetivo:

- **Fluxo em etapas:** foi adotado um fluxo em passos (criar viagem, definir orçamento, registrar gastos, visualizar resumo), em vez de telas únicas com muitos campos, para evitar sobrecarga de informação.
- **Informações essenciais em cards e listas:** optou-se por exibir viagens, orçamentos e gastos em cards e listas simples, evitando dashboards muito complexos com excesso de gráficos e indicadores.
- **Organização por viagem e design limpo:** a navegação foi organizada a partir de cada viagem (detalhes, itinerário, orçamento e gastos), com layout limpo e foco em legibilidade dos valores, em vez de uma interface carregada ou centrada apenas em transações isoladas.

3.2.7 Critérios de Escalabilidade, Resiliência e Segurança

A solução será desenvolvida de forma desacoplada entre interface, API e banco de dados, o que facilita a evolução da aplicação e ajustes futuros de infraestrutura, mantendo a arquitetura simples e organizada.

Em termos de resiliência, o backend fará validações de entrada e tratamento de erros nas operações principais, retornando respostas padronizadas para o frontend e evitando que falhas simples resultem em travamentos da aplicação.

No aspecto de segurança, serão adotadas boas práticas como armazenamento de senhas com hash, autenticação baseada em tokens, limitação dos dados pessoais coletados, uso preferencial de HTTPS no ambiente de produção e validações no backend para reduzir vulnerabilidades comuns em aplicações web.

3.3 FERRAMENTAS TECNOLÓGICAS

3.3.1 Linguagens de Programação

As principais linguagens e tecnologias utilizadas no desenvolvimento do Planit Go serão:

- **JavaScript com React (frontend):** Será utilizada a biblioteca React para construir a interface web do sistema em JavaScript. O React permite criar interfaces de usuário a partir de componentes reutilizáveis, facilitando a organização do código e a manutenção da aplicação, além de ser amplamente adotado na construção de interfaces modernas (React, 2025).
- **JavaScript com Node.js (backend):** No backend, será utilizado Node.js como ambiente de execução JavaScript no servidor, em conjunto com o framework Express para criação da API. O Node.js é um runtime assíncrono, de código aberto e multiplataforma, adequado para aplicações web baseadas em requisições HTTP, permitindo que frontend e backend utilizem a mesma linguagem (Node.js, 2025).
- **SQL com PostgreSQL (banco de dados relacional):** Para o armazenamento dos dados, será utilizado um banco de dados relacional PostgreSQL. O PostgreSQL é um SGBD open source amplamente utilizado, que oferece suporte a transações, integridade referencial e linguagem SQL padrão, sendo adequado para modelar entidades como usuários, viagens, categorias de gasto, itinerário e registros de despesas de forma estruturada (PostgreSQL Global Development Group, 2025a).

3.3.2 Frameworks e Bibliotecas

Os principais frameworks e bibliotecas utilizados no desenvolvimento do Planit Go serão:

- **Express.js (backend):** Framework minimalista para Node.js, utilizado para criação da API do sistema. O Express simplifica o tratamento de rotas HTTP, middleware e respostas da aplicação, sendo amplamente utilizado para construção de APIs web de forma rápida e flexível (Express, 2025).
- **JSON Web Token (JWT) - biblioteca jsonwebtoken:** Biblioteca utilizada para geração e validação de tokens de autenticação. O JWT permite que o servidor emita um token assinado após o login, que será enviado pelo cliente nas requisições subsequentes, dispensando o uso de sessão no servidor e facilitando o controle de acesso a rotas protegidas (jsonwebtoken, 2023).
- **Axios (cliente HTTP):** Biblioteca utilizada no frontend para realizar requisições HTTP à API do Planit Go. O Axios é um cliente HTTP baseado em Promises, compatível com navegador e Node.js, facilitando o envio de dados, tratamento de respostas e interceptação de requisições (Axios, 2025).

- **React Router (roteamento no frontend):** Biblioteca de roteamento para React, utilizada para gerenciar a navegação entre as telas da aplicação sem recarregar a página. Com o React Router, é possível definir rotas para páginas como login, lista de viagens e detalhes de uma viagem, mantendo a URL sincronizada com o conteúdo exibido (React Router, 2025).
- **PWA (Progressive Web App):** O frontend do Planit Go será desenvolvido seguindo princípios de Progressive Web App, permitindo que a aplicação seja acessada pelo navegador como um site responsivo e, em navegadores compatíveis, possa ser instalada na tela inicial do dispositivo por meio de um manifesto web. Essa abordagem melhora a experiência do usuário em dispositivos móveis, aproximando o comportamento do sistema ao de uma aplicação nativa (MDN Web Docs, 2025).
- **Material UI (componentes de interface):** Biblioteca de componentes para React baseada no Material Design, utilizada para agilizar a criação da interface do usuário com elementos visuais padronizados (botões, campos, diálogos, cards, etc.), garantindo uma aparência consistente e responsiva com menor esforço de estilização manual (MUI, 2025).

3.3.3 Ferramentas de Desenvolvimento e Gestão

As principais ferramentas de desenvolvimento e gestão utilizadas no projeto Planit Go serão:

- **Visual Studio Code (VS Code):** Editor de código-fonte utilizado no desenvolvimento do frontend e do backend. O VS Code foi escolhido por ser leve, extensível e oferecer integração nativa com Git, terminal embutido e um ecossistema amplo de extensões para JavaScript, Node.js e ferramentas de desenvolvimento web (Microsoft, 2025).
- **Git e GitHub:** O Git será utilizado como sistema de controle de versão e o GitHub como repositório remoto do código-fonte. Essa combinação permite registrar o histórico de alterações, criar branches para novas funcionalidades e facilitar o acompanhamento da evolução do projeto (Software Freedom Conservancy, 2025; GitHub, 2025b).
- **GitHub Actions (CI/CD):** O GitHub Actions será usado para configurar fluxos básicos de integração contínua, como execução de scripts de build e testes automatizados a cada push ou pull request. Isso contribui para verificar a qualidade do código de forma automatizada e apoiar futuras rotinas de entrega contínua (GitHub, 2025a).
- **Docker:** O Docker poderá ser utilizado para criar contêineres com o ambiente do backend e do banco de dados, facilitando a padronização do ambiente de desenvolvimento e a implantação em outros servidores quando necessário (Docker Inc., 2025).

- **Postman:** O Postman será utilizado para testar manualmente as rotas da API, validar os endpoints do backend e inspecionar requisições e respostas HTTP antes e durante a integração com o frontend (Postman, 2025).
- **Plataforma de nuvem (ex.: AWS):** Para publicação do sistema, será utilizado um serviço de hospedagem em nuvem, como a Amazon Web Services (AWS), permitindo que a aplicação backend e o banco de dados sejam acessados remotamente por meio da internet. A escolha da nuvem facilita o acesso ao sistema para fins de validação e apresentação do projeto (Amazon Web Services, 2025).

3.3.4 Licenciamento

As principais bibliotecas utilizadas no Planit Go são baseadas em licenças de software livres e permissivas, que permitem uso, modificação e distribuição do código, desde que respeitados os termos de cada licença.

- **Licença MIT:** As bibliotecas React, Express.js, Axios, React Router e Material UI são distribuídas sob a licença MIT, uma licença de software permissiva que permite o uso, cópia, modificação e redistribuição do software, com poucas restrições, desde que o aviso de copyright e os termos da licença sejam mantidos (Open Source Initiative, 2025).
- **Licença PostgreSQL:** O banco de dados PostgreSQL é distribuído sob a PostgreSQL License, uma licença open source permissiva, semelhante às licenças BSD e MIT, que permite uso, modificação e redistribuição do software, inclusive em aplicações comerciais (PostgreSQL Global Development Group, 2025b).

3.4 CONSIDERAÇÕES DE SEGURANÇA

3.4.1 Riscos Identificados

Os principais riscos de segurança considerados para o Planit Go são:

- Falhas de autenticação e controle de acesso, como uso de senhas fracas ou acesso a rotas protegidas sem validação adequada.
- Injeção de código no banco de dados (injeção SQL) por meio de entradas maliciosas em formulários e parâmetros de requisição.
- Validação insuficiente de dados de entrada, gerando comportamentos inesperados ou abertura de brechas na aplicação.
- Exposição indevida de dados pessoais de usuários, seja em respostas da API ou em mensagens de erro detalhadas.

3.4.2 Medidas de Mitigação

Para mitigar esses riscos, serão adotadas as seguintes ações:

- Uso de autenticação baseada em tokens (JWT) e armazenamento de senhas com hash, protegendo o acesso às rotas restritas.
- Utilização de consultas parametrizadas e boas práticas de acesso ao banco de dados, reduzindo a possibilidade de injeção de código.
- Aplicação de validações no backend para todos os dados recebidos da interface, conferindo tipos, tamanhos e formatos antes do processamento.
- Retorno apenas das informações necessárias em cada resposta da API, evitando exposição de dados sensíveis em erros ou logs.

3.4.3 Normas e Boas Práticas Seguidas

O desenvolvimento do Planit Go seguirá boas práticas de engenharia de software e segurança, tomando como referência:

- As normas legais já mencionadas neste documento, em especial a LGPD e o Marco Civil da Internet, para tratamento de dados pessoais e uso responsável de informações de usuários.
- As recomendações do OWASP Top 10 para aplicações web, em especial no que se refere à prevenção de falhas de autenticação, injeção de código e exposição de dados sensíveis, servindo como guia para validação de entradas e proteção de credenciais.
- Princípios gerais de gestão de segurança da informação, como controle de acesso, proteção de informações e redução de riscos por meio de políticas e procedimentos básicos de segurança adequados ao contexto do projeto.

3.4.4 Responsabilidade Ética

O uso dos dados será restrito às funcionalidades do sistema, não havendo compartilhamento indevido com terceiros. Serão observados princípios de transparência, consentimento e privacidade, alinhando o projeto a um uso responsável das informações dos usuários.

3.5 CONFORMIDADE E NORMAS APLICÁVEIS

- **LGPD (Lei nº 13.709/2018):** coleta apenas de dados pessoais necessários, uso com base em consentimento e possibilidade de o usuário gerenciar seus próprios dados (visualizar, atualizar e excluir).

- **Marco Civil da Internet (Lei nº 12.965/2014):** respeito à privacidade e ao uso responsável dos dados de acesso, com uso preferencial de comunicação segura (HTTPS) em ambiente de produção.
- **OWASP Top 10:** uso das recomendações como referência para evitar vulnerabilidades comuns em aplicações web, especialmente falhas de autenticação, injeção de código e exposição de dados sensíveis. Serão aplicadas medidas básicas, como autenticação via JWT, uso de HTTPS em produção e consultas parametrizadas para proteger as informações.

4 PRÓXIMOS PASSOS

Esta seção apresenta os principais marcos do projeto e uma visão geral dos passos seguintes após a conclusão da RFC, contemplando o Portfólio I e o Portfólio II.

4.0.1 Marcos de Entrega (Portfólio I)

Os principais marcos definidos para a fase da RFC são:

- **Marco 1 – Entrega da RFC completa:** RFC com todas as revisões solicitadas entregue até **15 de novembro, 23h59**.
- **Marco 2 – RFC assinada e avaliada:** RFC assinada e com as considerações de três professores até **28 de novembro, 23h59**.

4.0.2 Próximos Passos e Visão Geral do Portfólio II

Após a conclusão e aprovação da RFC, o desenvolvimento do sistema (Portfólio II) terá início em **janeiro de 2026** e seguirá, em linhas gerais, as seguintes etapas:

- **Planejamento detalhado do desenvolvimento:** revisão da RFC, refinamento dos requisitos e definição das tarefas técnicas de backend e frontend.
- **Implementação do backend:** desenvolvimento da API em Node.js/Express com autenticação, cadastro de usuários, viagens, orçamento e registro de gastos, utilizando PostgreSQL como banco de dados.
- **Implementação do frontend:** desenvolvimento das telas principais em React/PWA, incluindo fluxo de login, criação de viagem, definição de orçamento, registro de gastos e visualização de resumos.
- **Integração, testes e ajustes:** integração entre frontend e backend, realização de testes funcionais e correção dos problemas identificados.
- **Preparação para entrega final:** implantação em ambiente de demonstração, revisão da documentação e preparação da apresentação do sistema.

As datas específicas das etapas do Portfólio II serão definidas conforme o cronograma oficial da disciplina, tomando os marcos aqui descritos como referência para acompanhamento do andamento do projeto.

REFERÊNCIAS

- Amazon Web Services. **AWS Documentation**. 2025. Acesso em: 14 nov. 2025. Disponível em: <https://docs.aws.amazon.com/>. Citado na página 17.
- Axios. **Axios Documentation**. 2025. Acesso em: 13 nov. 2025. Disponível em: <https://axios-http.com/docs/intro>. Citado na página 15.
- BrowserStack. **Top Software Quality Testing Metrics: Types, Calculation, Examples**. 2025. Acesso em: 13 nov. 2025. Disponível em: <https://www.browserstack.com/guide/software-quality-metrics>. Citado na página 7.
- Docker Inc. **Docker Documentation**. 2025. Acesso em: 14 nov. 2025. Disponível em: <https://docs.docker.com/>. Citado na página 16.
- DOCS, MDN Web. **Progressive Web Apps**. 2025. Acesso em: 10 nov. 2025. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/Progressive_web_apps. Citado na página 6.
- ESTATISTICA, Instituto Brasileiro de Geografia e. **Gastos com turismo nacional aumentam 11,7% em 2024**. 2025. Acesso em: 12 nov. 2025. Disponível em: <https://agenciadenoticias.ibge.gov.br/agencia-noticias/2012-agencia-de-noticias/noticias/44624-gastos-com-turismo-nacional-aumentam-11-7-em-2024>. Citado na página 4.
- EXPERIAN. **Survey explores how travelers handle group travel spending**. 2025. Acesso em: 8 nov. 2025. Disponível em: <https://www.experian.com/blogs/ask-experian/survey-financial-stress-of-traveling-with-friends/>. Citado na página 4.
- Express. **Express - Node.js web application framework**. 2025. Acesso em: 13 nov. 2025. Disponível em: <https://expressjs.com/>. Citado na página 15.
- GitHub. **GitHub Actions Documentation**. 2025. Acesso em: 14 nov. 2025. Disponível em: <https://docs.github.com/actions>. Citado na página 16.
- GitHub. **GitHub Documentation**. 2025. Acesso em: 14 nov. 2025. Disponível em: <https://docs.github.com/>. Citado na página 16.
- jsonwebtoken. **jsonwebtoken npm package**. 2023. Acesso em: 13 nov. 2025. Disponível em: <https://www.npmjs.com/package/jsonwebtoken>. Citado na página 15.
- MDN Web Docs. **Progressive Web Apps (PWAs)**. 2025. Acesso em: 14 nov. 2025. Disponível em: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps. Citado na página 16.
- Microsoft. **Visual Studio Code Documentation**. 2025. Acesso em: 13 nov. 2025. Disponível em: <https://code.visualstudio.com/docs>. Citado na página 16.
- MUI. **Material UI - React components for faster and easier web development**. 2025. Acesso em: 13 nov. 2025. Disponível em: <https://mui.com/material-ui/>. Citado na página 16.
- Node.js. **Node.js Documentation**. 2025. Acesso em: 13 nov. 2025. Disponível em: <https://nodejs.org/en>. Citado na página 15.
- Open Source Initiative. **The MIT License**. 2025. Acesso em: 14 nov. 2025. Disponível em: <https://opensource.org/license/mit>. Citado na página 17.

PostgreSQL Global Development Group. **PostgreSQL Documentation**. 2025. Acesso em: 13 nov. 2025. Disponível em: <https://www.postgresql.org/docs/>. Citado na página 15.

PostgreSQL Global Development Group. **PostgreSQL License**. 2025. Acesso em: 14 nov. 2025. Disponível em: <https://www.postgresql.org/about/licence/>. Citado na página 17.

Postman. **Postman API Platform Documentation**. 2025. Acesso em: 14 nov. 2025. Disponível em: <https://learning.postman.com/>. Citado na página 17.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de Software: uma abordagem profissional**. 8. ed. Porto Alegre: AMGH, 2016. Citado na página 7.

React. **React Documentation**. 2025. Acesso em: 13 nov. 2025. Disponível em: <https://react.dev/>. Citado na página 15.

React Router. **React Router Documentation**. 2025. Acesso em: 13 nov. 2025. Disponível em: <https://reactrouter.com/>. Citado na página 16.

SENE, Bruna. **Viagens da Geracao Z: como planejar a aventura dos sonhos sem apertar o bolso**. 2025. Riconnect Rico. Acesso em: 12 nov. 2025. Disponível em: <https://riconnect.rico.com.vc/analises/viagens-da-geracao-z-como-planejar-a-aventura-dos-sonhos-sem-apertar-o-bolso/>. Citado na página 4.

Software Freedom Conservancy. **Git Documentation**. 2025. Acesso em: 14 nov. 2025. Disponível em: <https://git-scm.com/doc>. Citado na página 16.

Nota: 8,5 (de 0 a 10)

Considerações:

é citado PWA para o front, mas não vi sendo refletido na modelagem da arquitetura (ex. Workers, cache, ...)

distribuição das despesas não ficou claro como será feito:

- * qual algoritmo será usado
- * regras ou percentuais
- * se é customizável

bem como o tratamento referente ao itinerário:

- * Como o itinerário será estruturado (lista? timeline? cards?)
- * Se há dependência entre itens
- * Se permite reordenar dias
- Se integra com o orçamento



Assinatura do Avaliador

Nota: 9,0 (de 0 a 10) 

Considerações:

Avaliador(a): Profa. Janaína Fontana Biffi Duarte

Avaliação do Projeto: Planit Go (Web App de Planejamento de Viagens)

I. Comentários (Pontos Fortes)

Relevância e Justificativa bem delimitadas: O projeto aborda um problema real e documentado—a dificuldade em equilibrar o orçamento de viagem e a fragmentação do planejamento. A justificativa é clara ao propor uma solução que centraliza o itinerário e o controle financeiro em uma ferramenta simples. O contexto de crescimento do turismo nacional e a digitalização do planejamento reforçam a necessidade da solução.

II. Sugestões de Melhoria para o próximo Portfólio

1. Além dos aspectos pontuados pelo primeiro avaliador, sugiro incluir um mockup da tela de Relatórios de Planejamento (RF05), que é um dos principais objetivos, para demonstrar visualmente como será feita a comparação final entre o orçamento planejado e os gastos realizados. Isso solidificaria a promessa de valor do projeto.
2. Sugiro incluir uma visão do cronograma em formato de tabela.

JANAÍNA FONTANA BIFFI DUARTE

Assinatura do Avaliador

26/11/2025

Nota: 8,0 (de 0 a 10)

Considerações:

Não ficou claro para mim como o sistema funcionará. Será só para cadastro de dados?

Como será feito o orçamento planejado? No texto você diz que não usará API porque o sistema fará automaticamente mas não entendi como isso será feito.

Peço que faça um caso de uso por requisito para que fique mais compreensível. Pode ser para as principais ações do sistema, como a criação do orçamento. Se for bem descrito o fluxo principal e o alternativo explicarão como essa ação funcionará.

Edicarsia

Assinatura do Avaliador